

Destinations

Name	Page
Output Condition: Results Returned NoResults = 'false'	
 db NBS_MSGOUTE Query.....Database Message Extraction	339
Output Condition: Results Returned NoResults = 'false'	
 js Combine Query Responses.....JavaScript (v.1)	336
Output Condition: No Results Returned NoResults = 'true'	
 no-op to Sink.....No-operation	350

**no-op REH to Sink**

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR Polling NBS Updates Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink*.

Destinations

Name	Page
 ECLIR - ELR Route Error Handler Sink.....Sink	13

**no-op to Sink**

Filter

No-operation

On ECLIR - ELR Polling NBS Updates route

Notes

Purpose: This No-operation filter sends all messages to the *ECLIR - ELR Sink* after queries are completed or if empty result sets are received.

Destinations

Name	Page
 ECLIR - ELR Sink.....Sink	14



ECLIR - ELR Validation

Folder

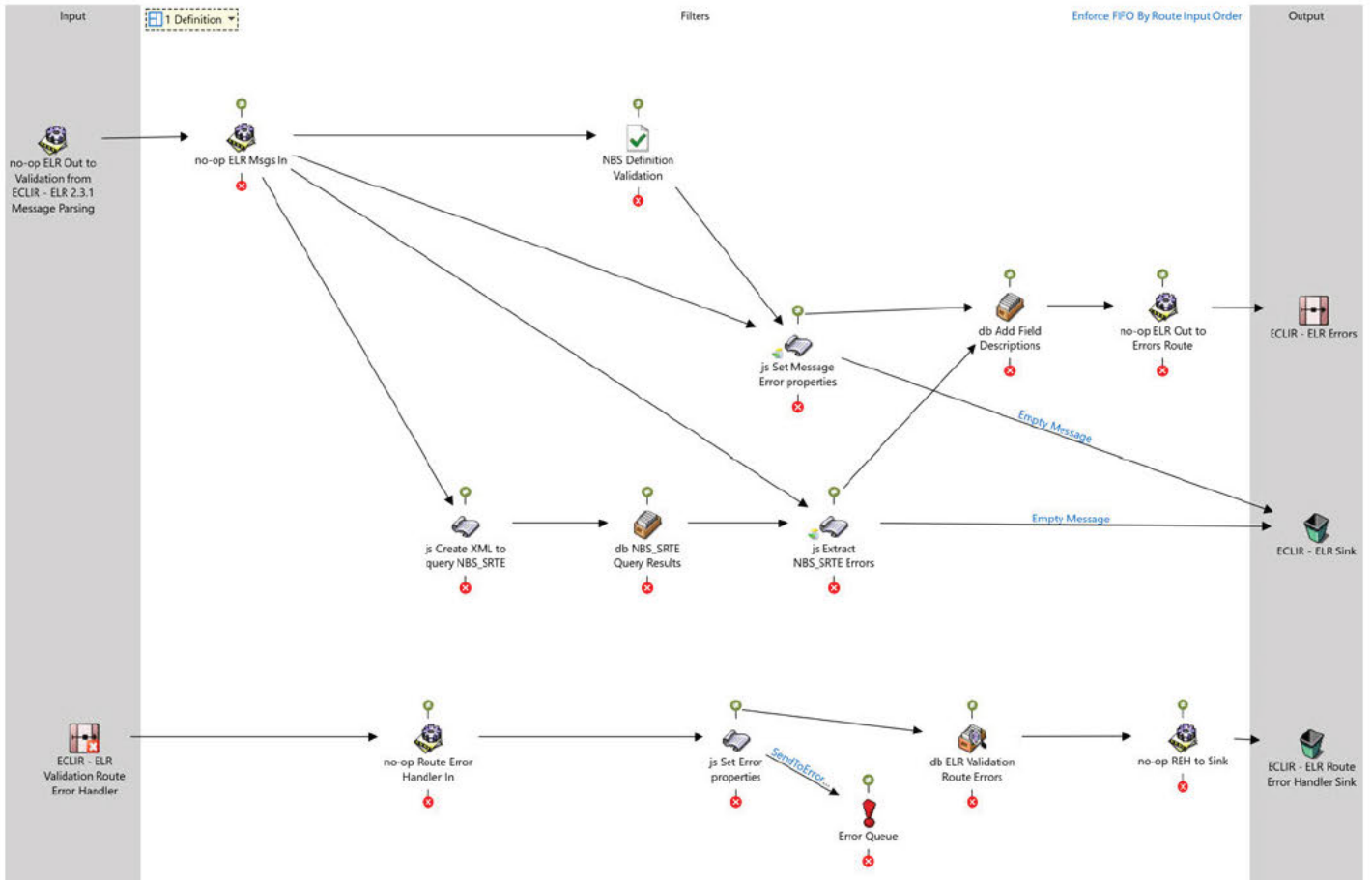
Full Path /ECLIR/Route__ECLIR/ELR/ECLIR - ELR Validation

Summary		
Routes (2)		
Folder Contents		Page
 ECLIR - ELR 2.3.1 Message Validation.....	Route	353
 ECLIR - ELR 2.5.1 Message Validation.....	Route	407



ECLIR - ELR 2.3.1 Message Validation

Route



Notes

Purpose: This route receives HL7 v2.3.1 messages from the *ECLIR - ELR 2.3.1 Message Parsing* route. The data in the messages is validated against the following specifications and any violations will be sent to the *ECLIR - ELR Errors* route to be recorded in the ECLIR ELR_ERROR table:

ECLIR_ELORU_2.3.1_NBS_validation.s3d message definition

NBS ODS processing information, conditional predicate and conformance rules

[Redacted]

Inputs

no-op ELR Out to Validation (*filter from "ECLIR - ELR 2.3.1 Message Parsing"*).....No-operation 232

Outputs

ECLIR - ELR Errors.....Route 157

ECLIR - ELR Sink.....Sink 14

ECLIR - ELR Route Error Handler Sink.....Sink 13

Filters

Name	Page
 db Add Field Descriptions.....Database Message Extraction	402
 db ELR Validation Route Errors..... Database LookUp	403
 db NBS_SRTE Query Results..... Database Message Extraction	379
 Error Queue..... Error Queue	383
 js Create XML to query NBS_SRTE..... JavaScript (v.1)	383
 js Extract NBS_SRTE Errors..... JavaScript (v.1)	392
 js Set Error properties..... JavaScript (v.1)	355
 js Set Message Error properties..... JavaScript (v.1)	357
 NBS Definition Validation.....HL7 Acknowledgement Generation	405
 no-op ELR Msgs In..... No-operation	382
 no-op ELR Out to Errors Route..... No-operation	356
 no-op REH to Sink..... No-operation	383
 no-op Route Error Handler In..... No-operation	379

Details

General	
Priority	Normal
Startup State	Startup Third
FIFO Mode	Enforce FIFO By Route Input Order
Stop route on error message	False



js Set Error properties

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Validation route

Filter

Notes

Purpose: This JavaScript filter

Sets the property `VALIDATION_TYPE` with the type of validation error to be stored in the `VALIDATION_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `ERROR_CATEGORY` with the category of the error to be stored in the `ERROR_CATEGORY` column in the `ELR_ERROR` table. The category is set to 'FATAL' because the message stops processing.

Sets the property `ERROR_TYPE` with the type of error to be stored in the `ERROR_TYPE` column in the `ELR_ERROR` table. The type is set to 'Rhapsody' because the error is an internal Rhapsody error.

Sets the property `PROCESSING_STEP` with the Rhapsody route name and filter name to be stored in the `PROCESSING_STEP` column in the `ELR_ERROR` table.

Sets the property `ERROR_MSG` with the Rhapsody generated errors to be stored in the `ERROR_MSG` column in the `ELR_ERROR` table.

Sets the property `ENVIRONMENT` with the Rhapsody Variable `ECLIR_ENVIRONMENT` to be stored in the `ENVIRONMENT` column in the `ELR_MESSAGE` table.

Sets the content of the message back to the original to be stored in the `ORIGINAL_MESSAGE` column in the `ELR_MESSAGE` table if needed.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);

//
=====
// _____ INITIAL VARIABLES (end
)
//
=====

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (start
)
//
=====

next.setProperty('VALIDATION_TYPE', 'Rhapsody');
next.setProperty('ERROR_CATEGORY', 'FATAL');
next.setProperty('ERROR_TYPE', 'Rhapsody');
```

Javascript

```

next.setProperty('PROCESSING_STEP', 'Parsing: ECLIR - ELR 2.3.1 Message Validation (Route) ->'
+ next.getProperty('rhapsody:ErrorFilterName') + ' (Filter)');
//next.setProperty('ERROR_MSG', 'Internal Rhapsody Route Error. Message processing could not
continue: ' + next.getErrors());
//next.setProperty('SendToErrorQueue', false);
var ERROR_MSG = 'Internal Rhapsody Route Error. Message processing could not continue: ' + next
.getErrors();
next.setProperty('ERROR_MSG', ERROR_MSG);

//Send the message to the Error Queue if it was caused by a database connectivity issue.
if (ERROR_MSG.indexOf('The connection is broken and recovery is not possible.') != -1 || ERROR
_MSG == 'Internal Rhapsody Route Error. Message processing could not continue: ') {
    next.setProperty('SendToErrorQueue', true);
}

//This is a Rhapsody variable that should be set up during configuration.
var [REDACTED]_ENVIRONMENT = getVariable("ECLIR_[REDACTED]_ENVIRONMENT");
if ([REDACTED]_ENVIRONMENT == null || [REDACTED]_ENVIRONMENT == "") {
    [REDACTED]_ENVIRONMENT = "VARIABLE NOT SETUP IN RHAPSODY";
}
next.setProperty('[REDACTED]_ENVIRONMENT', [REDACTED]_ENVIRONMENT);

// Load the snapshot back into the message.
if (next.getProperty("SnapshotIDofOrigMsg") != null) {
    next.text = next.loadMessageSnapshotForId(next.getProperty("SnapshotIDofOrigMsg")).text;
}

//
=====
// _____ SET MESSAGE PROPERTIES FOR DB INSERT (end
// _____
//
=====

```

Destinations

Name	Page
Output Condition	
SendToErrorQueue = 'true' ignoring case	
 Error Queue..... Error Queue	383
Error Connector	Page
 db ELR Validation Route Errors..... Database LookUp	403

**no-op ELR Out to Errors Route**

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: This No-operation filter sends XML formatted error messages to the *ECLIR - ELR Errors* route.

Notes

Destinations

Name	Page
 no-op ELR Validation Errors In..... No-operation	159



js Set Message Error properties

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: In this JavaScript filter, each section of the JavaScript code traverses each segment and ordered group according to the message definition ECLIR_ELOR_ORU_2.3.1_NBS_validation.s3d for an ORU – Unsolicited Observation Message (Event R01).

A message collector is used to input 2 messages into the filter.

The first message is the original HL7 v2 ELR message.

The second message is the HL7 v2 ACK message received from the *NBS Definition Validation* HL7 Acknowledgement Generation filter.

These 2 messages are then evaluated according to the following sections of code:

NBS definition errors. Any errors returned from the *NBS Definition Validation* HL7 Acknowledgement Generation filter are written to the output XML file to be stored in the ECLIR database.

NBS processing information, conformance and conditional predicate checks. If a field is repeated but NBS only stores the first value, an informational error record is stored. Specific fields in the message are checked for conformance or presence if they are required because of a condition. Any violations are written to the output XML file to be stored in the ECLIR database.

Conformance and conditional predicate checks. Specific fields in the message are checked for conformance or presence if they are required because of a condition. Any violations are written to the output XML file to be stored in the ECLIR database.

This filter outputs an XML file that follows the ECLIR - Error.xsd format for use to write these errors to the ECLIR database in the *ECLIR - ELR Errors* route.

Javascript

```
//
=====
// _____ INITIAL VARIABLES (start
)_____
//
=====
var next = output.append(input[0]);
var ELRMessage = input[0];
var ackNBS = input[1];
var errList = '';
var dbInsertsXML = new XML(<message></message>);

//
=====
```

Javascript

```
// _____ INITIAL VARIABLES (end)
)
//
=====

//
=====
// _____ FUNCTIONS (start)
)
//
=====

//FUNCTION: fnCreateXMLErrorRecord
//Function to create an XML node for writing an error record to the ELR_ERRORS table.
// recErrorTypeDetail parameter is used to combine errors into one record if the same rule was
// violated by multiple validations.
// Missing data
// Invalid date
// Invalid date/timeframe
// Invalid SSN
// Invalid format - LOINC
// Invalid format - date
// Invalid format - alpha
// Invalid format - numeric
// Invalid format - email
// Invalid format - zip
// Invalid format - ISO
// Invalid format - CLIA
// Invalid type - ISO
// Invalid type - CLIA
// Field must equal value
// Invalid value
// Field length
// Extra data
// Other
function fnCreateXMLErrorRecord (recValidationType, recErrorCategory, recFieldNotation, rec
PathHL7, recErrorMsg, recErrorType, recErrorTypeDetail) {
  tempEle = new XML(<errorRecord></errorRecord>);
  dbColumnNode = <VALIDATION_TYPE>{fnTruncateToMax(recValidationType, 50)}</VALIDATION_TYPE>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <ELR_MESSAGE_ID>{next.getProperty('ELR_MESSAGE_ID')}</ELR_MESSAGE_ID>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <ERROR_CATEGORY>{fnTruncateToMax(recErrorCategory, 50)}</ERROR_CATEGORY>;
  tempEle.appendChild(dbColumnNode);
  dbColumnNode = <PROCESSING_STEP>{'Validation: ECLIR - ELR Validation (Route):js Set Message
Error properties (Filter)'}</PROCESSING_STEP>;
  tempEle.appendChild(dbColumnNode);
  if (recFieldNotation != '') {
    dbColumnNode = <FIELD>{fnTruncateToMax(recFieldNotation, 150)}</FIELD>;
    tempEle.appendChild(dbColumnNode);
  }
  dbColumnNode = <FIELD_PATH>{fnTruncateToMax(recPathHL7, 1000)}</FIELD_PATH>;
  tempEle.appendChild(dbColumnNode);
  if (recPathHL7 != null && recPathHL7 != '') {
```

Javascript

```

var colFIELD_VALUE = ELRMessage.getField(recPathHL7);
if (colFIELD_VALUE != null) {
    if (colFIELD_VALUE.length > 1000) {
        colFIELD_VALUE = '(***Field Value truncated. Please see original message.) ' + colFIELD_VALUE;
    }
    dbColumnName = <FIELD_VALUE>{fnTruncateToMax(colFIELD_VALUE, 1000)}</FIELD_VALUE>;
    tempEle.appendChild(dbColumnName);
}
dbColumnName = <ERROR_MSG>{recErrorMsg}</ERROR_MSG>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE>{fnTruncateToMax(recErrorType, 50)}</ERROR_TYPE>;
tempEle.appendChild(dbColumnName);
dbColumnName = <ERROR_TYPE_DETAIL>{fnTruncateToMax(recErrorTypeDetail, 75)}</ERROR_TYPE_DETAIL>;
tempEle.appendChild(dbColumnName);
dbInsertsXML.appendChild(tempEle);
}

//FUNCTION: fnTruncateToMax
//Function to truncate the colValue to the max length intMax.
function fnTruncateToMax(colValue, intMax) {
    if (colValue != null) {
        var strField = colValue.substring(0, intMax);
        return strField;
    }
}

//FUNCTION: fnCheckForSimilarErrors
//Function to check if a similar error was previously flagged for the same field.
function fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation, pathHL7, error
Msg, errorType, errorTypeDetail) {
    var otherErrors = false;
    //Check to see if a duplicate error was flagged by NIST or NBS validation.
    if (dbInsertsXML != '' && dbInsertsXML != null) {
        next.text = dbInsertsXML;
        if ((next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL
='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE") != null)) {
            var valType = next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR
_TYPE_DETAIL='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE");
            if (valType.indexOf(valDefinition) == -1) { valType += ', ' + valDefinition; }

            //Update validation type with new value concatenated.
            next.setField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL='" +
errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/VALIDATION_TYPE", valType);

            //Check if the ERROR_CATEGORY is a higher severity. If so, set to higher severity.
            var valCategory = next.getField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR
_TYPE_DETAIL='" + errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/ERROR_CATEGORY");
            if (errorCategory != valCategory) {
                if ((errorCategory == 'FATAL') || (errorCategory == 'WARNING' && valCategory == 'INFO')) {

                    next.setField("/message/errorRecord[FIELD_PATH='" + pathHL7 + "' and ERROR_TYPE_DETAIL='"
+ errorTypeDetail + "' and ERROR_TYPE='" + errorType + "']/ERROR_CATEGORY", errorCategory);
                }
            }
        }
    }
}

```

Javascript

```

    }
    otherErrors = true;
  }
  dbInsertsXML = next.xml;
}
if (!otherErrors) { fnCreateXMLErrorRecord(valDefinition, errorCategory, fieldNotation, path
HL7, errorMsg, errorType, errorTypeDetail); }
}

//FUNCTION: fnValidateDataTypeRule
//Function to validate a field based on it's data type and conformance rules.
function fnValidateDataTypeRule (dataType, checkLOINC, dtValidationType, dtErrorCategory, dt
FieldNotation, dtPathHL7) {
  switch(dataType) {
    case 'Required':
      var errorMsg = 'This message is missing ' + dtFieldNotation + ' (' + dtPathHL7.substr(dt
PathHL7.lastIndexOf('/')+1) + ') which is a required field.';

      if (ELRMessage.getField(dtPathHL7) == null) {
        fnCheckForSimilarErrors(dtValidationType, dtErrorCategory, dtFieldNotation, dtPathHL7,
errorMsg, 'CONFORMANCE', 'Missing data');
      }
      default:
        break;
  }
}

//FUNCTION: fnValidateFieldEqualsValue
//Function to check a conformance rule that a field equals a specific value.
function fnValidateFieldEqualsValue (valDefinition, errorCategory, fieldNotation, pathHL7, req
Value, fieldToEqualNotation, pathFieldToEqual) {
  var fieldEqualsValue = false;
  var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must be
';
  var reqValues;
  if (reqValue != null) {
    reqValues = reqValue.split('_||_');
    for (var i=0; i < reqValues.length; i++) {
      if (ELRMessage.getField(pathHL7) == reqValues[i]) { fieldEqualsValue = true; }
    }
  }
  else if (ELRMessage.getField(pathHL7) == null) { fieldEqualsValue = true; }

  if (!fieldEqualsValue) {
    if (fieldToEqualNotation != '') {
      errorMsg += 'the same value as ' + fieldToEqualNotation + ' (' + pathFieldToEqual.substr
(pathFieldToEqual.lastIndexOf('/')+1) + '). The ' + fieldNotation
+ ' value is "' + ELRMessage.getField(pathHL7) + '". The ' + fieldToEqualNotation + '
value is "' + ELRMessage.getField(pathFieldToEqual) + '".';
    }
    else {
      if (reqValue != null) {
        if (reqValues.length > 1) {
          reqValue = reqValues.join(', ');
        }
      }
    }
  }
}

```

Javascript

```

        errorMsg += 'one of the values "' + reqValue + '". The value in this message is "' + ELRM
essage.getField(pathHL7) + '".';
    }
    else {
        errorMsg += 'the value "' + reqValue + '". The value in this message is "' + ELRMessage
.getField(pathHL7) + '".';
    }
}
}
fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Field must equal value');
}
}
}

```

[illegible]

Javascript

```

        if (fieldValue.match(/^\d{4}(\d{2})?(\d{2})?(\d{2})?(\d{2})?(\d{2})?\.\.?\\d?\\d?\\d?(\\+|-) [0-9]
    ] [0-9]{3})?) == null) {
            formatRep = 'YYYY[MM][DD][HH[MM[SS[.S[S[S[S]]]]]]][+/-ZZZZ]';
            invalidFormat = true;
        }
        if (!invalidFormat && fieldValue.match(/\\d{4}|(?:?:?:?:?:?:[13579][26]| [2468][048])00|
    (?::[0-9]{2}(?:?:?:[13579][26])|(?::[2468][048]|0[48])))(?:?:?:?:09|04|06|11)(?::0[1-9]|1[0-9]|2[0-9]
    |30))|(?::(?:01|03|05|07|08|10|12)(?::0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?::0[1-9]|1[0-9]|2[0-9]
    |30)))|(?::[0-9]{4})(?::(?:?:09|04|06|11)(?::0[1-9]|1[0-9]|2[0-9]|30))|(?::(?:01|03|05|07|08|10|12)
    (?::0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?::[01][0-9]|2[0-8])))|(?::0[0-9]|1[0-9]|2[0-3])(?:((?::[0-5]
    [0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-) [0-1] [0-9]{3})?) == null) {
            invalidDate = true;
        }
        break;
    case 'TS_ELR_TimeOpt':
        if (fieldValue.match(/\\d{4}\\d{2}\\d{2}(\\d{2})?(\\d{2})?(\\d{2})?\\.\\.?\\d?\\d?\\d?(\\+|-) [0-1] [0-9]
    {3})?) == null) {
            formatRep = 'YYYYMMDD[HH[MM[SS[.S[S[S[S]]]]]]][+/-ZZZZ]';
            invalidFormat = true;
        }
        if (!invalidFormat && fieldValue.match(/(?:?:?:?:?:?:[13579][26]| [2468][048])00|(?::[0-9]
    ]{2}(?:?:?:[13579][26])|(?::[2468][048]|0[48])))(?:?:?:?:09|04|06|11)(?::0[1-9]|1[0-9]|2[0-9]|30
    ))|(?::(?:01|03|05|07|08|10|12)(?::0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?::0[1-9]|1[0-9]|2[0-9]
    |30)))|(?::[0-9]{4})(?::(?:?:09|04|06|11)(?::0[1-9]|1[0-9]|2[0-9]|30))|(?::(?:01|03|05|07|08|10|12)
    (?::0[1-9]|1[0-9]|2[0-9]|3[01]))|(?:02(?::[01][0-9]|2[0-8])))|(?::0[0-9]|1[0-9]|2[0-3])(?:((?::[0-5]
    [0-9]){2})?(\\.\\d)?\\d?\\d?\\d?(\\+|-) [0-1] [0-9]{3})?) == null) {
            invalidDate = true;
        }
        break;
    }
}

if (invalidDate) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must
have a valid date. The value in this message "' + ELRMessage.getField(pathHL7) + '" is not a
valid date.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Invalid date');
}
if (invalidFormat) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must
follow the required date format of "' + formatRep + '". The value in this message "' + ELRM
essage.getField(pathHL7) + '" does not follow the correct format.';
    fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Invalid format - date');
}
}
}

//FUNCTION: fnValidateWithinTimeFrame
//Function to validate if date was within the valid time frame and not a future date.
function fnValidateWithinTimeFrame (valDefinition, errorCategory, fieldNotation, pathHL7, valid
TimeFrame) {
    var fieldDate = ELRMessage.getField(pathHL7);
    if ((fieldDate != null) && (fieldDate.length >= 8)) {

```

Javascript

```

    fieldDate = new Date(Number(fieldDate.substr(0,4)),Number(fieldDate.substr(4,2))-1,Number
(fieldDate.substr(6,2)));
    // Calculate the date of the time frame.
    var todaysDate = new Date();
    todaysDate = new Date(todaysDate.getFullYear(),todaysDate.getMonth(),todaysDate.getDate());
    var pastDate = new Date(todaysDate);
    pastDate.setDate(pastDate.getDate() - validTimeFrame);
    if ((fieldDate < pastDate) || (fieldDate > todaysDate)) {
        var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') should
be within the past ' + validTimeFrame +
        ' days and cannot be in the future. The value in this message "' + ELRMessage.getField
(pathHL7) + '" is not within the correct time frame.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg, 'CON
FORMANCE', 'Invalid date/timeframe');
    }
}
}

//FUNCTION: fnIsValidUniversalIDType
//Function to validate the correct Universal ID Type is present.
function fnIsValidUniversalIDType (valDefinition, errorCategory, fieldNotation, pathHL7,
idType) {
    var errorMsg = 'The field ' + fieldNotation + ' (UniversalIDType) does not contain the correct
value. ';
    switch(idType) {
        case 'ISO':
            if (ELRMessage.getField(pathHL7) != 'ISO')
            {
                errorMsg += ' "ISO" is the only accepted value. The value in this message is "' + ELRM
essage.getField(pathHL7) + '".';
                fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid type - ISO');
            }
            break;
        case 'ISOCLIA':

            errorMsg += ' "CLIA" or "ISO" are the only accepted values. The value in this message is
"' + ELRMessage.getField(pathHL7) + '".';
            if ((ELRMessage.getField(pathHL7) != 'CLIA') && (ELRMessage.getField(pathHL7) != 'ISO')) {
                fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid type - CLIA');
            }
            break;
    }
}

//FUNCTION: fnRequiredIfOtherField
//Function to check predicate condition for if a field is required based on another field's
value.
function fnRequiredIfOtherField (valDefinition, errorCategory, fieldNotation_ReqField, field
Notation_OtherField, path_ReqField, path_OtherField, otherSpecificValue, validationLogic) {
    var errorMsg = 'This message is missing ' + fieldNotation_ReqField + ' (' + path_ReqField
.substr(path_ReqField.lastIndexOf('/')+1) + ')' + ' which is required if ' + fieldNotation
_OtherField + ' (' + path_OtherField.substr(path_OtherField.lastIndexOf('/')+1) + ')';
    switch(validationLogic) {
        case 'IsAnyValue':

```

Javascript

```

//The field is required if the other field is any value.
if (ELRMessage.getField(path_OtherField) != null && ELRMessage.getField(path_ReqField) ==
null) {
    errorMsg += ' is valued.';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
}
break;
case 'IsNotValued':
//The field is required if the other field is not valued.
if (ELRMessage.getField(path_OtherField) == null && ELRMessage.getField(path_ReqField) ==
null) {
    errorMsg += ' is not valued.';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
}
break;
case 'IsSpecificValue':
//The field is required if the other field equals a specific value.
if (ELRMessage.getField(path_OtherField) != otherSpecificValue && ELRMessage.getField(path
_ReqField) == null) {
    errorMsg += ' equals "' + otherSpecificValue + '".';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
}
break;
case 'IsNotSpecificValue':
//The field is required if the other field does not equal a specific value.
if (ELRMessage.getField(path_OtherField) != otherSpecificValue && ELRMessage.getField(path
_ReqField) == null) {
    errorMsg += ' does not equal "' + otherSpecificValue + '".';
    fnCheckForSimilarErrors (valDefinition, errorCategory, fieldNotation_ReqField, path_Req
Field, errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
}
break;
default:
break;
}
}

//FUNCTION: fnRequiredIfTwoFieldsMissing
//Function to check if a field is present if it is required when two other fields are missing.
function fnRequiredIfTwoFieldsMissing (valDefinition, errorCategory, path_ReqField, pathHL7_Mis
singData1, pathHL7_MissingData2, msgErrorLoc_ReqField, msgErrorLoc_MissingData1, msgErrorLoc
_MissingData2, fieldNotation) {
    if ((ELRMessage.getField(pathHL7_MissingData1) == null) && (ELRMessage.getField(pathHL7_Mis
singData2) == null) && (ELRMessage.getField(path_ReqField) == null)) {
        var errorMsg = 'This message is missing ' + msgErrorLoc_ReqField + ' (' + path_ReqField
.substr(path_ReqField.lastIndexOf('/')+1) + ')' + ' which is required if '
+ msgErrorLoc_MissingData1 + ' (' + pathHL7_MissingData1.substr(pathHL7_MissingData1.last
IndexOf('/')+1) + ')'
+ ' and ' + msgErrorLoc_MissingData2 + ' (' + pathHL7_MissingData2.substr(pathHL7_Missing
Data2.lastIndexOf('/')+1) + ')' + ' are not valued.';
        fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, path_ReqField, errorMsg,
'CONDITIONAL PREDICATE', 'Missing data');
    }
}
}

```

Javascript

```
//FUNCTION: fnValidateFormat
//Function to validate various formats.
function fnValidateFormat (valDefinition, errorCategory, fieldNotation, pathHL7, checkNull,
  fieldReqLength, invalidPattern, validationFormat) {
  var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') must ';
  var fieldValue = ELRMessage.getField(pathHL7);
  if (fieldValue != null || validationFormat == 'AlphaOnly') {
    switch(validationFormat) {
      case 'ISO':
        if (fieldValue.match(/^([0-2])(\.\d)|(\.[1-9][0-9]*)*)$/)) == null) {
          errorMsg += 'follow the required ISO OID format. The value in this message "' + field
Value + '" does not follow the correct format.';
          fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - ISO');
        }
        break;
      case 'CLIA':
        if (fieldValue.match(/[0-9]{2}D[0-9]{7}/)) == null) {
          errorMsg += 'be a valid CLIA identifier format. The value in this message "' + fieldValue
+ '" does not follow the correct format.';
          fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - CLIA');
        }
        break;
      case 'ZipCode':
        if ((fieldValue != null) && (fieldValue.match(/^d{5}(-d{4})?/)) == null)) {
          errorMsg += 'be formatted as 99999[-9999] for US Zip or ZIP +4 codes. The value in this
message "' + fieldValue + '" does not follow the correct format.';
          fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - zip');
        }
        break;
      case 'LOINC':
        errorMsg += 'be a valid LOINC code identifier format. The value in this message "' + field
Value + '" does not follow the correct format.';
        if (fieldValue.match(/^[a-zA-Z0-9]{1,8}-d{1}/)) == null) {
          fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - LOINC');
        }
        break;
      case 'SSN':
        if (fieldValue.match(/(?!(\d)\1{8})(?!(\d)\2\2(\d)\2(\d)\2{3})(?:?!000|666|9|123\d\d)\d{3}
(?:00|45)\d\d(?:0000|6789)\d{4})/)) == null) {
          errorMsg += 'be formatted as ##### (9 digits, no special characters) and cannot be all
the same number, cannot be 123-45-6789. The value in this message "' + fieldValue + '" does
not follow the correct format.';
          fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid SSN');
        }
        break;
      case 'AlphaOnly':
        var isValid = true;
        errorMsg += 'contain alpha characters only and cannot be null. The value in this message
"' + fieldValue + '" does not follow the correct format.';
        if (checkNull) {
          if ((fieldValue == null) || (fieldValue == '')) { isValid = false; }
        }
    }
  }
}
```

Javascript

```

    }
    if (fieldValue != null) {
        if (fieldValue.match(/^([a-zA-Z]+)$/)) == null) { isValid = false; }
    }

    if (!isValid) { fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, path
HL7, errorMsg, 'CONFORMANCE', 'Invalid format - alpha'); }
    break;
    case 'Numeric':
        var isValid = true;
        var regex = new RegExp("^([\\d]){" + fieldReqLength.toString() + "}$");

        if (fieldReqLength != '') {
            if ((fieldValue.match(regex) == null) || (fieldValue == invalidPattern)) {
                isValid = false;
                errorMsg += 'be a length of ' + fieldReqLength.toString() + ', must ';
            }
        }
        else {
            if (fieldValue.match(/[\\d]*/)) == null) { isValid = false; }
        }
        errorMsg += 'be numeric with no special characters, cannot be all the same number and
cannot be "' + invalidPattern + '". The value in this message "' + fieldValue + '" does not
follow the correct format.';
        if (!isValid) {
            fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - numeric'); }
        break;
    case 'Email':
        errorMsg += 'be a valid email format (localpart@domain). The value in this message "' +
fieldValue + '" does not follow the correct format.';
        if (fieldValue.match(/^\\w+@[a-zA-Z_]+?\\.([a-zA-Z]{2,3})$/)) == null) {
            fnCheckForSimilarErrors(valDefinition, errorCategory, fieldNotation, pathHL7, errorMsg,
'CONFORMANCE', 'Invalid format - email');
        }
        break;
    } // -----(END) switch(validationFormat) (END)
} // -----(END) if (fieldValue != null) (END)
}

//FUNCTION: fnOBX4Required
//Function to determine if OBX-4 is a required field.
function fnOBX4Required (pathHL7, fieldValue, fieldIndex) {
    var arrOBX3_1 = [];
    for(var k=0; k < ELRMessage.getRepeatCount(pathHL7 + '/OBSERVATION'); k++) {
        var OBX = pathHL7 + '/OBSERVATION[' + k + ']/OBX';
        if (k != fieldIndex) { arrOBX3_1.push(ELRMessage.getField(OBX + '/ObservationIdentifier/Ide
ntifier')); }
    }
    if (arrOBX3_1.indexOf(fieldValue) != -1) {
        return true;
    }
}

//FUNCTION: fnInfoNotTreatedAsRepeatable

```

Javascript

```

//Function to create an info message if a field has repeating values and is not treated as repeatable.
function fnInfoNotTreatedAsRepeatable (fieldNotation, pathHL7) {
  if ((ELRMessage.getField(pathHL7) != null) && (ELRMessage.getRepeatCount(pathHL7) > 1)) {
    var errorMsg = fieldNotation + ' (' + pathHL7.substr(pathHL7.lastIndexOf('/')+1) + ') has multiple values. NBS does not treat this field as repeatable. If more than one value is sent in the message, then only the first occurrence is stored.';
    fnCreateXMLErrorRecord ('NBS', 'INFO', fieldNotation, pathHL7, errorMsg, 'ODS PROCESSING COMMENT', 'Extra data');
  }
}

//FUNCTION: fnCreateErrorMsg
//Function to create an error message based on the definition error code.
function fnCreateErrorMsg (errorCode, errorInitial, errorPath, fieldNotation) {
  var returnMsg = '';
  switch(errorCode) {
    case '1001': //Required segment missing
      returnMsg = 'This message is missing ' + errorPath.substr((errorPath.lastIndexOf('/')+1), 3) + ' which is a required segment.';
      break;
    case '1006': //Required field missing
      returnMsg = 'This message is missing ' + fieldNotation + ' (' + errorPath.substr(errorPath.lastIndexOf('/')+1) + ') which is a required field.';
      break;
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      if (ELRMessage.getField(errorPath) == null) { var fieldLength = 0; }
      else { var fieldLength = ELRMessage.getField(errorPath).length; }
      returnMsg = fieldNotation + ' (' + errorPath.substr(errorPath.lastIndexOf('/')+1) + ') has a max limit of ' + errorInitial.substr(errorInitial.indexOf('>')+2) + ' characters. The value in this message "'
        + ELRMessage.getField(errorPath) + '" is ' + fieldLength + ' characters.';
      break;
    case '1014': //Field value not found in validation table
      returnMsg = 'The code/value in field ' + fieldNotation + ' (' + errorPath.substr(errorPath.lastIndexOf('/')+1)
        + ') was not found in the corresponding validation table. The code/value in this message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1009': //Extra data found after field
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] | [[/]] [A-Z] {2} \d [[]]);
      returnMsg = 'There is extra data in the ' + errorPath.substr(indexSegment+1, 3) + ' segment found after the field "' + errorPath.substr(errorPath.lastIndexOf('/')+1) + '". The data in this message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1010': //Extra data found after composite
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] | [[/]] [A-Z] {2} \d [[]]);
      returnMsg = 'There is extra data in the ' + errorPath.substr(indexSegment+1, 3) + ' segment for the field "' + errorPath.substr(errorPath.lastIndexOf('/')+1) + '". The data in this message was "' + ELRMessage.getField(errorPath) + '".';
      break;
    case '1022': //Extra data found after segment
      var indexSegment = errorPath.search(/[[/]] [A-Z] {3} [[/]] | [[/]] [A-Z] {2} \d [[/]] | [[/]] [A-Z] {3} [[] | [[/]] [A-Z] {2} \d [[]]);

```

Javascript

```

    returnMsg = 'There is extra data at the end of the ' + errorPath.substr(errorPath.lastIndexOf('/')+1, 3) + ' segment.';
    break;
  default:
    returnMsg = 'This message has a definition violation of ' + errorInitial + '.';
    break;
  }
  return returnMsg;
}

```

```

//FUNCTION: fnCreateErrorTypeDetail
//Function to create an error type detail based on the definition error code.
function fnCreateErrorTypeDetail (errorCode) {
  var returnDetail = '';
  switch(errorCode) {
    case '1001': //Required segment missing
    case '1006': //Required field missing
      returnDetail = 'Missing data';
      break;
    case '1007': //Field too short
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      returnDetail = 'Field length';
      break;
    case '1014': //Field value not found in validation table
      returnDetail = 'Invalid value';
      break;
    case '1009': //Extra data found after field
    case '1010': //Extra data found after composite
    case '1022': //Extra data found after segment
      returnDetail = 'Extra data';
      break;
    default:
      returnDetail = 'Other';
      break;
  }
  return returnDetail;
}

```

```

//FUNCTION: fnCreateErrorCategory
//Function to create an error category based on the definition error code.
function fnCreateErrorCategory (errorCode) {
  var returnCategory = '';
  switch(errorCode) {
    case '1001': //Required segment missing
      returnCategory = 'FATAL';
      break;
    case '1006': //Required field missing
    case '1007': //Field too short
    case '1008': //Field too long
    case '1013': //Field too long, truncated
      returnCategory = 'WARNING';
      break;
    default:
      returnCategory = 'INFO';
      break;
  }
}

```

Javascript

```

    }
    return returnCategory;
}

//
=====
//_____ FUNCTIONS (end
)
//
=====

//
=====
//_____ NBS Definition Errors (start
)
//
=====
//Parse out all validation errors from the NBS Definition ACK generated and create an ELR
_ERRORS db record.

//NHBS messages are not sent to NBS.
if (next.getProperty('nhbs_SendingFac') != null) { var nhbs_SendingFac = next.getProperty
('nhbs_SendingFac').toLowerCase(); }
if (next.getProperty('nhbs_PatientLName') != null) { var nhbs_PatientLName = next.getProperty
('nhbs_PatientLName').toLowerCase(); }

if (!(nhbs_SendingFac == 'cphl') && (nhbs_PatientLName == 'nhbs')) {

    if (ackNBS.getField('ERR') != null) {
        for (var i=0; i < ackNBS.getRepeatCount('ERR/ErrorCodeAndLocation'); i++) {
            var errFieldLocation = '';
            if (ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/SegmentID') != null) { errField
Location = ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/SegmentID'); }
            if (ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/FieldPosition') != null) { err
FieldLocation += '-' + ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/FieldPosition'); }

            var errPathHL7 = '';
            if (ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/CodeIdentifyingError/Alternate
Text') != null) { errPathHL7 = ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/CodeIden
tifyingError/AlternateText'); }

            var errorCategory = fnCreateErrorCategory(ackNBS.getField('ERR/ErrorCodeAndLocation[' + i +
']/CodeIdentifyingError/Identifier'));
            //Some specific fields are FATAL errors instead of being based on the error code.
            switch(errFieldLocation) {
                case 'MSH-1':
                case 'MSH-2':
                case 'MSH-4':
                case 'MSH-10':
                case 'OBR-3':
                    errorCategory = 'FATAL';
                    break;
            }
        }
    }
}

```

Javascript

```

    var errorTypeDetail = fnCreateErrorTypeDetail(ackNBS.getField('ERR/ErrorCodeAndLocation[' +
    i + ']/CodeIdentifyingError/Identifier'));
    var errorMsg = fnCreateErrorMsg(ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/Code
    IdentifyingError/Identifier'),
        ackNBS.getField('ERR/ErrorCodeAndLocation[' + i + ']/CodeIdentifyingError/Text'),
        errPathHL7, errFieldLocation);

    fnCreateXMLErrorRecord ('NBS', errorCategory, errFieldLocation, errPathHL7, errorMsg, 'DEF
    INITION', errorTypeDetail);
  }
}

//
=====
//_____ NBS Definition Errors (end
)
//
=====

//
=====
//_____ NBS Predicates and Conformance (start
)
//
=====

//Validate if all conditional fields have had conditions met
// and validate content of fields if a conformance rule exists.

//----- PATIENT_RESULT group -----
for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
  var PID = 'PATIENT_RESULT[' + i + ']/PATIENT/PID';

  //----- PID segment -----
  fnInfoNotTreatedAsRepeatable ('PID-13', PID + '/PhoneNumberHome');
  fnInfoNotTreatedAsRepeatable ('PID-14', PID + '/PhoneNumberBusiness');
  fnInfoNotTreatedAsRepeatable ('PID-15', PID + '/PrimaryLanguage');
  fnInfoNotTreatedAsRepeatable ('PID-21', PID + '/MothersIdentifier');
  //----- PID segment (end) -----

  if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NK1') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1'); j++) {
      var NK1 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']';

      //----- NK1 segment -----
      fnInfoNotTreatedAsRepeatable ('NK1-2', NK1 + '/Name');
      fnInfoNotTreatedAsRepeatable ('NK1-3.1', NK1 + '/Relationship/Identifier');
      fnInfoNotTreatedAsRepeatable ('NK1-4', NK1 + '/Address');
      fnInfoNotTreatedAsRepeatable ('NK1-5', NK1 + '/PhoneNumber');
      //----- NK1 segment (end) -----
    }
  }
}

```

Javascript

```

}

//----- ORDER_OBSERVATION group -----
for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION'); j++) {
    var ORC = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/ORC';
    var OBR = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBR';

    //----- ORC segment -----
    if ((ELRMessage.getField(ORC + '/OrderingProvider') == null) && (ELRMessage.getField(ORC + '/OrderingFacilityName') == null)) {
        var errorMsg = 'Either Ordering Facility (ORC-21) or Ordering Provider (ORC-12) must be present in the message. Both fields in the message are empty.';
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-12', ORC + '/OrderingProvider', errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-21', ORC + '/OrderingFacilityName', errorMsg, 'CONDITIONAL PREDICATE', 'Missing data');
    }
    fnInfoNotTreatedAsRepeatable ('ORC-21', ORC + '/OrderingFacilityName');
    fnInfoNotTreatedAsRepeatable ('ORC-22', ORC + '/OrderingFacilityAddress');
    fnInfoNotTreatedAsRepeatable ('ORC-23', ORC + '/OrderingFacilityPhoneNumber');
    fnInfoNotTreatedAsRepeatable ('ORC-24', ORC + '/OrderingProviderAddress');
    //----- ORC segment (end) -----

    //----- OBR segment -----
    fnInfoNotTreatedAsRepeatable ('OBR-10', OBR + '/CollectorIdentifier');
    if ((ELRMessage.getField(OBR + '/OrderingProvider') == null) && (ELRMessage.getField(ORC + '/OrderingFacilityName') == null)) {
        var errorMsg = 'Either Ordering Facility (ORC-21) or Ordering Provider (OBR-16) must be present in the message. Both fields in the message are empty.';
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'OBR-16', OBR + '/OrderingProvider', errorMsg, 'CONFORMANCE', 'Missing data');
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'ORC-21', ORC + '/OrderingFacilityName', errorMsg, 'CONFORMANCE', 'Missing data');
    }
    fnInfoNotTreatedAsRepeatable ('OBR-16', OBR + '/OrderingProvider');
    fnInfoNotTreatedAsRepeatable ('OBR-17', OBR + '/OrderCallbackPhoneNumber');
    //----- OBR segment (end) -----

    //----- OBSERVATION group -----
    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) {
        var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/OBX';

        //----- OBX segment -----
        if ((ELRMessage.getField(OBX + '/ObservationSubID') == null) && (fnOBX4Required('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']', ELRMessage.getField(OBX + '/ObservationIdentifier/Identifier'), k))) {
            var errorMsg = 'This message is missing OBX-4 (ObservationSubID) which is required if there are multiple OBX segments associated with the same OBR segment that have the same OBX-3 values for (OBX-3.1 and OBX-3.3) or (OBX-3.4 and OBX-3.6).';
            fnCreateXMLErrorRecord ('NBS', 'WARNING', 'OBX-4', OBX + '/ObservationSubID', errorMsg, 'CONFORMANCE', 'Missing data');
        }
    }
}

```

Javascript

```

    if ((ELRMessage.getField(OBX + '/Units') == null) && (ELRMessage.getField(OBX + '/Value
Type') == 'SN' || ELRMessage.getField(OBX + '/ValueType') == 'NM')) {
        var errorMsg = 'This message is missing OBX-6 (Units) which is a required field if OBX-2
is valued "NM" or "SN" AND OBX-11 is not valued "X".';
        fnCreateXMLErrorRecord ('NBS', 'WARNING', 'OBX-6', OBX + '/Units', errorMsg, 'CONDITIONAL
PREDICATE', 'Missing data');
    }
    fnInfoNotTreatedAsRepeatable ('OBX-8', OBX + '/AbnormalFlags');
    //----- OBX segment (end) -----

    } // -----(END) for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) (END)

    //----- OBSERVATION group (end) -----

    } // -----(END) for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION'); j++) (END)
    //--- ORDER_OBSERVATION group (end) ---

    } // -----(END) for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
    //---- PATIENT_RESULT group (end) ----

    //
    =====
    // _____ NBS Predicates and Conformance (end
    )
    //
    =====

    //
    =====
    // _____ Conformance Rules (start
    )
    //
    =====
    //Validate against all _____ conformance rules.

    //----- MSH segment -----
    fnValidateFieldEqualsValue('_____', 'INFO', 'MSH-1', 'MSH/FieldSeparator', '|', '', '');
    if (ELRMessage.getField('MSH/SendingFacility/UniversalIDType') == 'CLIA') { fnValidateFormat
('_____', 'INFO', 'MSH-4.2', 'MSH/SendingFacility/UniversalID', '', '', '', 'CLIA'); }
    if (ELRMessage.getField('MSH/SendingFacility/UniversalIDType') == 'ISO') { fnValidateFormat
('_____', 'INFO', 'MSH-4.2', 'MSH/SendingFacility/UniversalID', '', '', '', 'ISO'); }
    fnIsValidUniversalIDType('_____', 'INFO', 'MSH-4.3', 'MSH/SendingFacility/UniversalIDType', 'ISO
CLIA');
    fnValidateFieldEqualsValue('_____', 'INFO', 'MSH-5', 'MSH/ReceivingApplication', '_____'ELR^2.16
.840.1.113883.3.773.1.1.3^ISO', '', '');
    fnValidateFieldEqualsValue('_____' 'INFO', 'MSH-6', 'MSH/ReceivingFacility', '_____'2.16.840.1
.113883.3.773^ISO', '', '');
    fnValidateWithinTimeFrame('_____', 'INFO', 'MSH-7', 'MSH/DateTimeOfMessage', 30);

    if (ELRMessage.getField('MSH/VersionID') == '2.5.1') {
        fnValidateFieldEqualsValue ('_____', 'INFO', 'MSH-9.1', 'MSH/MessageType/MessageCode', 'ORU',
'', '');
        fnValidateFieldEqualsValue ('_____', 'INFO', 'MSH-9.2', 'MSH/MessageType/TriggerEvent', 'R01',
'', '');
    }

```

Javascript

```

fnValidateFieldEqualsValue ( [REDACTED], 'INFO', 'MSH-9.3', 'MSH/MessageType/MessageStructure', 'ORU
R01', '', '');
}

if ((ELRMessage.getField('MSH/VersionID').indexOf('2.3.1') == -1) && (ELRMessage.getField('MSH/
VersionID').indexOf('2.5.1') == -1)) {
    var errorMsg = 'For MSH-12 (VersionID), "2.3.1" or "2.5.1." are the only accepted values
("2.5.1" is required for Promoting Interoperability Program).';
    fnCheckForSimilarErrors([REDACTED], 'INFO', 'MSH-12', 'MSH/VersionID', errorMsg, 'CONFORMANCE',
'Field must equal value');
}
if (ELRMessage.getField('MSH/AcceptAcknowledgmentType') != null) { fnValidateFieldEqualsValue
([REDACTED], 'INFO', 'MSH-15', 'MSH/AcceptAcknowledgmentType', 'NE', '', ''); }
if (ELRMessage.getField('MSH/ApplicationAcknowledgmentType') != null) { fnValidateFieldEquals
Value([REDACTED], 'INFO', 'MSH-16', 'MSH/ApplicationAcknowledgmentType', 'NE', '', ''); }
//----- MSH segment (end) -----

//----- PATIENT_RESULT group -----
for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) {
    var PID = 'PATIENT_RESULT[' + i + ']/PATIENT/PID';

    //----- PID segment -----
    if (ELRMessage.getField(PID + '/PatientIdentifierList') != null) {
        for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientIdentifierList'); j++) {
            var PID3 = PID + '/PatientIdentifierList[' + j + ']';
            if ((ELRMessage.getField(PID3 + '/IdentifierTypeCode') == 'SS') || (ELRMessage.getField
(PID3 + '/AssigningAuthority/UniversalIDType') == 'SS')) {
                fnValidateFormat([REDACTED], 'INFO', 'PID-3.1', PID3 + '/IDNumber', '', '', '', 'SSN');
            }
        }
    }
    fnValidateDataTypeRule([REDACTED] Required', false, [REDACTED], 'INFO', 'PID-5', PID + '/PatientName');
    if (ELRMessage.getField(PID + '/PatientName') != null) {
        for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientName'); j++) {
            var PID5 = PID + '/PatientName[' + j + ']';
            fnValidateFormat([REDACTED], 'INFO', 'PID-5.1', PID5 + '/FamilyName', true, '', '', 'AlphaOnly');
            fnValidateFormat([REDACTED], 'INFO', 'PID-5.2', PID5 + '/GivenName', true, '', '', 'AlphaOnly');
            fnValidateFormat([REDACTED], 'INFO', 'PID-5.3', PID5 + '/SecondandFurtherGivenNamesorInitials
Thereof', false, '', '', 'AlphaOnly');
            fnValidateFieldEqualsValue([REDACTED], 'INFO', 'PID-5.7', PID5 + '/NameTypeCode', 'L', '', '');
        }
    }

    //PID-7 (DateTimeOfBirth)
    if (ELRMessage.getField(PID + '/DateTimeOfBirth') != null) {
        fnValidateDateTime('TS_ELR_TimeOpt', [REDACTED], 'INFO', 'PID-7', PID + '/DateTimeOfBirth');

        //Date of birth cannot be before 01/01/1910 and not in the future.
        var fieldDate = ELRMessage.getField(PID + '/DateTimeOfBirth');
        if ((fieldDate != null) && (fieldDate.length >= 8) && (fieldDate.match(/\d{8}/) != null)) {
            fieldDate = new Date(Number(fieldDate.substr(0,4)),Number(fieldDate.substr(4,2))-1,Number
(fieldDate.substr(6,2)));
            var pastDate = new Date(1910, 0, 1);
            var todaysDate = new Date();
            todaysDate = new Date(todaysDate.getFullYear(),todaysDate.getMonth()+11,todaysDate.get
Date());

```

Javascript

```

    if ((todaysDate < fieldDate) || (fieldDate < pastDate)) {
        var errorMsg = "PID-7 (DateTimeOfBirth) with value '" + ELRMessage.getField(PID + '/Date
TimeOfBirth') + "' cannot be before 01/01/1910 and not in the future.";
        fnCheckForSimilarErrors('■■■■', 'INFO', 'PID-7', PID + '/DateTimeOfBirth', errorMsg, 'CON
FORMANCE', 'Invalid date');
    } // (END) if ((todaysDate < fieldDate) || (fieldDate < pastDate)) (END)
} // (END) if ((fieldDate != null) && (fieldDate.length >= 8) && (fieldDate.match(/\d{8}/)
!= null)) (END)
} // (END) if (ELRMessage.getField(PID + '/DateTimeOfBirth') != null) (END)
// (END) PID-7 (DateTimeOfBirth) (END)

fnValidateDataTypeRule ('■■■■Required', false, '■■■■', 'INFO', 'PID-8', PID + '/Administrative
Sex');

fnValidateDataTypeRule ('■■■■Required', false, '■■■■', 'INFO', 'PID-11', PID + '/PatientAddress
');
if (ELRMessage.getField(PID + '/PatientAddress') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PatientAddress'); j++) {
        var PID11 = PID + '/PatientAddress[' + j + ']';
        if (ELRMessage.getField(PID11 + '/City') != null) { fnValidateFormat ('■■■■', 'INFO', 'PID
-11.3', PID11 + '/City', false, '', '', 'AlphaOnly'); }
        if (ELRMessage.getField(PID11 + '/ZiporPostalCode') != null) { fnValidateFormat ('■■■■',
'INFO', 'PID-11.5', PID11 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
    }
}

fnValidateDataTypeRule ('■■■■Required', false, '■■■■', 'INFO', 'PID-13', PID + '/PhoneNumber
Home');
if (ELRMessage.getField(PID + '/PhoneNumberHome') != null) {
    for (var j=0; j < ELRMessage.getRepeatCount(PID + '/PhoneNumberHome'); j++) {
        var PID13 = PID + '/PhoneNumberHome[' + j + ']';
        if (ELRMessage.getField(PID13 + '/EmailAddress') != null) { fnValidateFormat ('■■■■',
'INFO', 'PID-13.4', PID13 + '/EmailAddress', '', '', '', 'Email'); }
        if (ELRMessage.getField(PID13 + '/AreaCityCode') != null) { fnValidateFormat ('■■■■',
'INFO', 'PID-13.6', PID13 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
        if (ELRMessage.getField(PID13 + '/LocalNumber') != null) { fnValidateFormat ('■■■■',
'INFO', 'PID-13.7', PID13 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
        if (ELRMessage.getField(PID13 + '/Extension') != null) { fnValidateFormat ('■■■■', 'INFO',
'PID-13.8', PID13 + '/Extension', '', '', '', 'Numeric'); }
    }
}

fnValidateDataTypeRule ('■■■■Required', false, '■■■■', 'INFO', 'PID-22', PID + '/EthnicGroup');
if (ELRMessage.getField(PID + '/PatientDeathDateandTime') != null) { fnValidateWithinTime
Frame('■■■■', 'INFO', 'PID-29', PID + '/PatientDeathDateandTime', 60); }

if (ELRMessage.getField(PID + '/PatientDeathIndicator') != null) { fnValidateFieldEquals
Value ('■■■■', 'INFO', 'PID-30', PID + '/PatientDeathIndicator', 'Y_|_|_N', '', ''); }
//----- PID segment (end) -----

if (ELRMessage.getField('PATIENT_RESULT[' + i + ']/PATIENT/NK1') != null) {
    //----- NK1 segment -----
    if (ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1') > 0) {
        for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1'); j++) {
            var NK1 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']';
            fnValidateFormat ('■■■■', 'INFO', 'NK1-1', NK1 + '/SetIDNK1', '', '', '', 'Numeric');
            if (ELRMessage.getField(NK1 + '/Name') != null) {

```

Javascript

```

    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j +
    ']/Name'); k++) {
        var NK1_2 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Name[' + k + ']';
        fnValidateFormat(██████ 'INFO', 'NK1-2.1', NK1_2 + '/FamilyName', false, '', '', 'Alpha
Only');
        fnValidateFormat(██████ 'INFO', 'NK1-2.2', NK1_2 + '/GivenName', false, '', '', 'Alpha
Only');
    }
}

    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j +
    ']/Address'); k++) {
        var NK1_4 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/Address[' + k + ']';
        if (ELRMessage.getField(NK1_4 + '/City') != null) { fnValidateFormat(██████ 'INFO',
        'NK1-4.3', NK1_4 + '/City', false, '', '', 'AlphaOnly'); }
        if (ELRMessage.getField(NK1_4 + '/ZiporPostalCode') != null) { fnValidateFormat(██████
        'INFO', 'NK1-4.5', NK1_4 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
    }

    for (var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j +
    ']/PhoneNumber'); k++) {
        var NK1_5 = 'PATIENT_RESULT[' + i + ']/PATIENT/NK1[' + j + ']/PhoneNumber[' + k + ']';
        if (ELRMessage.getField(NK1_5 + '/AreaCityCode') != null) { fnValidateFormat (██████
        'INFO', 'NK1_5.6', NK1_5 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
        if (ELRMessage.getField(NK1_5 + '/LocalNumber') != null) { fnValidateFormat ██████
        'INFO', 'NK1_5.7', NK1_5 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    }
}
//----- NK1 segment (end) -----
}

//----- ORDER_OBSERVATION group -----

for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION'); j+
+) {
    var ORC = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/ORC';
    var OBR = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBR';

    //----- ORC segment -----
    fnValidateDataTypeRule ('██████_Required', false, '██████', 'INFO', 'ORC-12', ORC + '/OrderingProv
ider');
    fnValidateFieldEqualsValue ('██████', 'INFO', 'ORC-12', ORC + '/OrderingProvider', ELRMessage
.getField(OBR + '/OrderingProvider'), 'OBR-16', OBR + '/OrderingProvider');

    if (ELRMessage.getField(ORC + '/OrderingProvider') != null) {
        for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProvider'); k++) {
            var ORC12 = ORC + '/OrderingProvider[' + k + ']';
            if (ELRMessage.getField(ORC12 + '/IdentifierTypeCode') == 'NPI') { fnValidateFormat
            ('██████ 'INFO', 'ORC-12.1', ORC12 + '/IDNumber', '', 9, '123456789', 'Numeric'); }
            if (ELRMessage.getField(ORC12 + '/FamilyName') != null) { fnValidateFormat(██████,
            'INFO', 'ORC-12.2', ORC12 + '/FamilyName', false, '', '', 'AlphaOnly'); }
            if (ELRMessage.getField(ORC12 + '/GivenName') != null) { fnValidateFormat(██████ 'INFO',
            'ORC-12.3', ORC12 + '/GivenName', false, '', '', 'AlphaOnly'); }
        }
    }
}

```

Javascrpt

```

if (ELRMessage.getField(ORC + '/CallBackPhoneNumber') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/CallBackPhoneNumber'); k++) {
    var ORC14 = ORC + '/CallBackPhoneNumber[' + k + ']';
    if (ELRMessage.getField(ORC14 + '/AreaCityCode') != null) { fnValidateFormat ('',
'INFO', 'ORC-14.6', ORC14 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(ORC14 + '/LocalNumber') != null) { fnValidateFormat ('',
'INFO', 'ORC-14.7', ORC14 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    if (ELRMessage.getField(ORC14 + '/Extension') != null) { fnValidateFormat ('',
'INFO', 'ORC-14.8', ORC14 + '/Extension', '', '', 'Numeric'); }
  }
}

//To determine jurisdiction for reporting: Patient address must be provided in [PID-11]
(Patient Address).
// However, if it is unknown and left empty, then the ordering provider must be documented.
// If both are unknown and left empty, then the ordering facility is required.
if ((ELRMessage.getField(PID + '/PatientAddress') == null) && ((ELRMessage.getField(ORC +
'/OrderingProvider') == null) || (ELRMessage.getField(OBR + '/OrderingProvider') == null)) &&
(ELRMessage.getField(ORC + '/OrderingFacilityName') == null)) {
  var errorMsg = 'To determine jurisdiction for reporting: Patient address must be provided
in PID-11 (Patient Address). However, if it is unknown and left empty, then the ordering pro
vider (ORC-12 or OBR-16) must be documented. If both are unknown and left empty, then the ord
ering facility (ORC-21) is required.';
  fnCreateXMLErrorRecord ('', 'INFO', 'PID-11', PID + '/PatientAddress', errorMsg, 'CONFORM
ANCE', 'Missing data');
}

fnValidateDataTypeRule ('Required', false, '', 'INFO', 'ORC-21', ORC + '/OrderingFaci
lityName');

fnValidateDataTypeRule ('Required', false, '', 'INFO', 'ORC-22', ORC + '/OrderingFaci
lityAddress');
if (ELRMessage.getField(ORC + '/OrderingFacilityAddress') != null) {
  fnValidateFormat ('', 'INFO', 'ORC-22.3', ORC + '/OrderingFacilityAddress/City', false,
'', '', 'AlphaOnly');
  fnValidateFormat ('', 'INFO', 'ORC-22.5', ORC + '/OrderingFacilityAddress/ZiporPostal
Code', '', '', 'ZipCode');
}

fnValidateDataTypeRule ('Required', false, '', 'INFO', 'ORC-23', ORC + '/OrderingFaci
lityPhoneNumber');
if (ELRMessage.getField(ORC + '/OrderingFacilityPhoneNumber') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingFacilityPhoneNumber'); k++) {
    var ORC23 = ORC + '/OrderingFacilityPhoneNumber[' + k + ']';
    if (ELRMessage.getField(ORC23 + '/EmailAddress') != null) { fnValidateFormat ('',
'INFO', 'ORC-23.4', ORC23 + '/EmailAddress', '', '', 'Email'); }
    if (ELRMessage.getField(ORC23 + '/AreaCityCode') != null) { fnValidateFormat ('',
'INFO', 'ORC-23.6', ORC23 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(ORC23 + '/LocalNumber') != null) { fnValidateFormat ('',
'INFO', 'ORC-23.7', ORC23 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    if (ELRMessage.getField(ORC23 + '/Extension') != null) { fnValidateFormat ('',
'INFO', 'ORC-23.8', ORC23 + '/Extension', '', '', 'Numeric'); }
  }
}

if (ELRMessage.getField(ORC + '/OrderingProviderAddress') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(ORC + '/OrderingProviderAddress'); k++) {
    var ORC24 = ORC + '/OrderingProviderAddress[' + k + ']';

```

Javascript

```

    if (ELRMessage.getField(ORC24 + '/City') != null) { fnValidateFormat('■■■■', 'INFO', 'ORC
-24.3', ORC24 + '/City', false, '', '', 'AlphaOnly'); }
    if (ELRMessage.getField(ORC24 + '/ZiporPostalCode') != null) { fnValidateFormat('■■■■',
'INFO', 'ORC-24.5', ORC24 + '/ZiporPostalCode', '', '', '', 'ZipCode'); }
  }
}
//----- ORC segment (end) -----

//----- OBR segment -----
fnValidateWithinTimeFrame('■■■■', 'INFO', 'OBR-7', OBR + '/ObservationDateTime', 30);
if (ELRMessage.getField(OBR + '/SpecimenReceivedDateTime') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/SpecimenReceivedDateTime'); k++) {
    fnValidateWithinTimeFrame('■■■■', 'INFO', 'OBR-14', OBR + '/SpecimenReceivedDateTime[' + k +
']', 30);
  }
}

fnValidateDataTypeRule ('■■■■Required', false, '■■■■', 'INFO', 'OBR-16', OBR + '/OrderingProv
ider');
fnValidateFieldEqualsValue ('■■■■', 'INFO', 'OBR-16', OBR + '/OrderingProvider', ELRMessage
.getField(ORC + '/OrderingProvider'), 'ORC-12', ORC + '/OrderingProvider');
if (ELRMessage.getField(OBR + '/OrderingProvider') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderingProvider'); k++) {
    var OBR16 = OBR + '/OrderingProvider[' + k + ']';
    if (ELRMessage.getField(OBR16 + '/IdentifierTypeCode') == 'NPI') { fnValidateFormat
('■■■■', 'INFO', 'OBR-16.1', OBR16 + '/IDNumber', '', 9, '123456789', 'Numeric'); }
    if (ELRMessage.getField(OBR16 + '/FamilyName') != null) { fnValidateFormat('■■■■',
'INFO', 'OBR-16.2', OBR16 + '/FamilyName', false, '', '', 'AlphaOnly'); }
    if (ELRMessage.getField(OBR16 + '/GivenName') != null) { fnValidateFormat('■■■■', 'INFO',
'OBR-16.3', OBR16 + '/GivenName', false, '', '', 'AlphaOnly'); }
  }
}

if (ELRMessage.getField(OBR + '/OrderCallbackPhoneNumber') != null) {
  for (var k=0; k < ELRMessage.getRepeatCount(OBR + '/OrderCallbackPhoneNumber'); k++) {
    var OBR17 = OBR + '/OrderCallbackPhoneNumber[' + k + ']';
    if (ELRMessage.getField(OBR17 + '/EmailAddress') != null) { fnValidateFormat ('■■■■',
'INFO', 'OBR-17.4', OBR17 + '/EmailAddress', '', '', '', 'Email'); }
    if (ELRMessage.getField(OBR17 + '/AreaCityCode') != null) { fnValidateFormat ('■■■■',
'INFO', 'OBR-17.6', OBR17 + '/AreaCityCode', '', 3, '123', 'Numeric'); }
    if (ELRMessage.getField(OBR17 + '/LocalNumber') != null) { fnValidateFormat ('■■■■',
'INFO', 'OBR-17.7', OBR17 + '/LocalNumber', '', 7, '1234567', 'Numeric'); }
    if (ELRMessage.getField(OBR17 + '/Extension') != null) { fnValidateFormat ('■■■■',
'INFO', 'OBR-17.8', OBR17 + '/Extension', '', '', '', 'Numeric'); }
  }
}

fnValidateFieldEqualsValue ('■■■■', 'INFO', 'OBR-25', OBR + '/ResultStatus', 'F_||_P_||_C',
'', '');
//----- OBR segment (end) -----

//----- OBSERVATION group -----
for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j
+ ']/OBSERVATION'); k++) {

```

Javascript

```

var OBX = 'PATIENT_RESULT[' + i + ']/ORDER_OBSERVATION[' + j + ']/OBSERVATION[' + k + ']/
OBX';

//----- OBX segment -----
fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-2', OBX + '/ValueType');
fnValidateFieldEqualsValue ('INFO', 'OBX-2', OBX + '/ValueType', 'SN_||_NM_||_CE_||
_CWE_||_ST_||_TX', '', '');

fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-3', OBX + '/Observation
Identifier');
fnValidateFieldEqualsValue ('INFO', 'OBX-3.3', OBX + '/ObservationIdentifier/Nameof
CodingSystem', 'LN', '', '');

fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-4', OBX + '/Observation
SubID');
if (ELRMessage.getField(OBX + '/ObservationSubID') != null) { fnValidateFormat ('
INFO', 'OBX-4', OBX + '/ObservationSubID', '', '', '', 'Numeric'); }

if ((ELRMessage.getField(OBX + '/ValueType') == 'SN') || (ELRMessage.getField(OBX + '/Value
Type') == 'NM')) {
fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-6', OBX + '/Units');
fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-7', OBX + '/References
Range');
}

fnValidateDataTypeRule ('Required', false, 'INFO', 'OBX-8', OBX + '/Abnormal
Flags');

fnValidateFieldEqualsValue ('INFO', 'OBX-11', OBX + '/ObservationResultStatus', 'F_||
_P_||_C', '', '');
//----- OBX segment (end) -----

} // -----(END) for(var k=0; k < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION[' + j + ']/OBSERVATION'); k++) (END)
//----- OBSERVATION group (end) -----

} // -----(END) for (var j=0; j < ELRMessage.getRepeatCount('PATIENT_RESULT[' + i + ']/
ORDER_OBSERVATION'); j++) (END)
//--- ORDER_OBSERVATION group (end) ---

} // -----(END) for (var i=0; i < ELRMessage.getRepeatCount('PATIENT_RESULT'); i++) (END)
//--- PATIENT_RESULT group (end) ---

//
=====
// _____ Conformance Rules (end)
)
//
=====

next.text = dbInsertsXML;

```

Message Collector: Input Collation	
Collection Order	[ECLIR - ELR 2.3.1 Message Validation/6fd292f8-d669-44e7-ba29-2cf12f762ade,ECLIR - ELR 2.3.1 Message Validation/c65755df-e855-4662-a81c-8122d88a36b9]
Match Property	ELR_MESSAGE_ID
Message Set Timeout	5
Timeout Units	Minutes
Timeout Destination	Process as Usual

Destinations

Name	Page
Output Condition: Empty Message	
Body does not contain 'errorRecord' ignoring case	
 ECLIR - ELR Sink..... Sink	14
Error Connector	Page
 db Add Field Descriptions..... Database Message Extraction	402



no-op Route Error Handler In

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: This No-operation filter receives messages from the *ECLIR - ELR 2.3.1 Message Validation Route Error Handler* when an unhandled error occurs anywhere in the route.

Destinations

Name	Page
 js Set Error properties..... JavaScript (v.1)	355



db NBS_SRTE Query Results

Filter

Database Message Extraction

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: This Database Lookup filter loops through the XML flat file created in the *js Create XML to query NBS_SRTE* JavaScript filter. All results found are written to an output XML file to be processed by the *js Extract NBS_SRTE Errors* JavaScript filter.

Notes

Configuration Properties

Maximum Concurrency	1	
Database		
Host		
Port		
Database Name		
Username		
Password		
Additional Connection Properties	Property Name	Property Value
Configuration File		
Message Body Column		
Message Type	XML	
Definition		
Query Timeout		
Deadlock Retry Attempts	5	
Deadlock Retry Delay	100	
Socket Timeout	0	
Configuration File:		

```

<statement name="Code_value_general" ide:type="executeSql" iterate="Code_value_general/query_record" ide:summary="Retrieves a record from the Code_value_general table if code is valid or if the description can be converted to a code." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Code_value_general/query_record/field_path`, '()') as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
  FROM Code_value_general
  WHERE code_set_nm = `/message/Code_value_general/query_record/code_set_nm`
  and (code = `/message/Code_value_general/query_record/code`
  or code_desc_txt = `/message/Code_value_general/query_record/code_desc_txt`
  or code_short_desc_txt = `/message/Code_value_general/query_record/code_short_desc_txt`
  or concept_code = `/message/Code_value_general/query_record/code`)</sql>
</statement>
<statement name="V_Race_code" ide:type="executeSql" iterate="V_Race_code/query_record" ide:summary="Retrieves a record from V_Race_code table if code is valid." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/V_Race_code/query_record/field_path`, '()') as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
  FROM V_Race_code
  WHERE code = `/message/V_Race_code/query_record/code`
  or concept_code = `/message/V_Race_code/query_record/code`</sql>
</statement>
<statement name="Language_code" ide:type="executeSql" ide:summary="Retrieves a record from Language_code table if code is valid." ide:allowCompoundStatements="true">

```

Configuration File:

```

<sql>SELECT QUOTENAME(`/message/Language_code/query_record/field_path`, '()') as field
_path, code_set_nm, code, code_desc_txt, code_short_desc_txt
FROM Language_code
WHERE code_set_nm = `/message/Language_code/query_record/code_set_nm`
and code = `/message/Language_code/query_record/code`</sql>
</statement>
<statement name="V_State_county_code_value" ide:type="executeSql" iterate="V_State_county
_code_value/query_record" ide:summary="Returns a record from V_State_county_code_value table
if code or description is valid." ide:allowCompoundStatements="true" user:iterationRootMessage
Path="message">
  <sql>SELECT QUOTENAME(`/message/V_State_county_code_value/query_record/field_path`, '()')
as field_path, code_set_nm, code, code_desc_txt, code_short_desc_txt, concept_code
FROM V_State_county_code_value
WHERE code_set_nm = `/message/V_State_county_code_value/query_record/code_set_nm`
and (code = `/message/V_State_county_code_value/query_record/code`
or code_desc_txt = `/message/V_State_county_code_value/query_record/code_desc_txt`
or code_short_desc_txt = `/message/V_State_county_code_value/query_record/code_short_desc
txt`
or concept_code = `/message/V_State_county_code_value/query_record/code`)</sql>
</statement>
<statement name="State_code" ide:type="executeSql" iterate="State_code/query_record" ide:sum
mary="Validate state code from code or text." ide:allowCompoundStatements="true" user:iteration
RootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/State_code/query_record/field_path`, '()') as field_path,
Code_value_general.code_set_nm, Code_value_general.code, Code_value_general.code_desc_txt,
Code_value_general.code_short_desc_txt, Code_value_general.concept_code, State_code.state_nm
FROM Code_value_general
FULL OUTER JOIN State_code
ON Code_value_general.code = State_code.state_cd
WHERE Code_value_general.code_set_nm = 'STATE_CCD'
and (Code_value_general.code = `/message/State_code/query_record/code`
or State_code.state_nm = `/message/State_code/query_record/code`
or Code_value_general.concept_code = `/message/State_code/query_record/code`)</sql>
</statement>
<statement name="Labtest_loinc" ide:type="executeSql" iterate="Labtest_loinc/query_record"
ide:summary="Returns a record from Labtest_loinc table if a local code can be translated to a
LOINC." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record/field_path`, '()') as field
_path, lab_test_cd, loinc_cd AS LocalToLOINC
FROM Labtest_loinc
WHERE status_cd = 'A' AND laboratory_id = `/message/Labtest_loinc/query_record/lab_id`
AND lab_test_cd = `/message/Labtest_loinc/query_record/lab_test_cd`</sql>
  <statement name="Check LOINC_condition" ide:type="executeSql" test="NotNull(@LocalToLOINC
)" ide:summary="If the local code has an equivalent LOINC, check table LOINC_condition to val
idate." ide:allowCompoundStatements="true">
    <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record[1]/field_path`, '()') as field
_path, loinc_cd
FROM LOINC_condition
WHERE loinc_cd = @LocalToLOINC</sql>
  </statement>
  <statement name="Check LOINC_code" ide:type="executeSql" test="NotNull(@LocalToLOINC)" ide
:summary="If the local code has an equivalent LOINC, check table LOINC_code to validate." ide
:allowCompoundStatements="true">
    <sql>SELECT QUOTENAME(`/message/Labtest_loinc/query_record[1]/field_path`, '()') as field
_path, loinc_cd
FROM LOINC_code
WHERE loinc_cd = @LocalToLOINC</sql>
  </statement>

```

Configuration File:

```

</statement>
<statement name="LOINC_condition" ide:type="executeSql" iterate="LOINC_condition/query_record" ide:summary="Returns the LOINC code if it is found in LOINC_condition table." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/LOINC_condition/query_record/field_path`, '()') as field_path, loinc_cd
  FROM LOINC_condition
  WHERE loinc_cd = `/message/LOINC_condition/query_record/loinc_cd`</sql>
</statement>
<statement name="LOINC_code" ide:type="executeSql" iterate="LOINC_code/query_record" ide:summary="Returns a record from LOINC_code table if LOINC code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/LOINC_code/query_record/field_path`, '()') as field_path, loinc_cd
  FROM LOINC_code
  WHERE loinc_cd = `/message/LOINC_code/query_record/loinc_cd`</sql>
</statement>
<statement name="Snomed_condition" ide:type="executeSql" iterate="Snomed_condition/query_record" ide:summary="Returns a record from Snomed_condition table if SNOMED code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Snomed_condition/query_record/field_path`, '()') as field_path, snomed_cd
  FROM Snomed_condition
  WHERE snomed_cd = `/message/Snomed_condition/query_record/snomed_cd`</sql>
</statement>
<statement name="Snomed_code" ide:type="executeSql" iterate="Snomed_code/query_record" ide:summary="Returns a record from Snomed_code table if SNOMED code exists." ide:allowCompoundStatements="true" user:iterationRootMessagePath="message">
  <sql>SELECT QUOTENAME(`/message/Snomed_code/query_record/field_path`, '()') as field_path, snomed_cd
  FROM Snomed_code
  WHERE snomed_cd = `/message/Snomed_code/query_record/snomed_cd`</sql>
</statement>
<statement name="dummy_record" ide:type="executeSql" ide:summary="This creates a dummy record in the output XML in the event none of the queries return a result." ide:allowCompoundStatements="true">
  <sql>SELECT SYSDATETIME() AS 'dummy_time_record'</sql>
</statement>
</config>

```

Destinations

Name	Page
js Extract NBS_SRTE Errors..... JavaScript (v.1)	392



no-op ELR Msgs In

No-operation

On ECLIR - ELR 2.3.1 Message Validation route

Filter

Notes

Purpose: This No-operation filter receives ELR messages from the *ECLIR - ELR 2.3.1 Message Parsing* route.

Notes

Destinations

Name		Page
js Create XML to query NBS_SRTE.....	JavaScript (v.1)	383
js Extract NBS_SRTE Errors.....	JavaScript (v.1)	392
js Set Message Error properties.....	JavaScript (v.1)	357
NBS Definition Validation.....	HL7 Acknowledgement Generation	405



Error Queue

Filter

Error Queue

On ECLIR - ELR 2.3.1 Message Validation route

Configuration Properties

Error Message	Sent from error filter
---------------	------------------------



no-op REH to Sink

Filter

No-operation

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: This No-operation filter sends messages from the *ECLIR - ELR 2.3.1 Message Validation Route Error Handler* to the *ECLIR - ELR Route Error Handler Sink* after the errors have been stored into the ELR_ERROR table in the ECLIR database.

Destinations

Name		Page
ECLIR - ELR Route Error Handler Sink.....	Sink	13



js Create XML to query NBS_SRTE

Filter

JavaScript (v.1)

On ECLIR - ELR 2.3.1 Message Validation route

Notes

Purpose: In this JavaScript filter:

Notes

The main JavaScript code traverses each segment and ordered group according to the message definition ECLIR_ELOR_ORU_2.3.1_NBS_validation.s3d for an ORU – Unsolicited Observation Message (Event R01). Data is parsed from the specific fields in the message that will be cross-referenced with a code set in a table in the NBS_SRTE database. This data, along with other relevant fields, is written to an XML flat file in order to easily iterate through elements to query the NBS_SRTE database in the *db NBS_SRTE Query Results* Database Lookup filter that follows.

Javascript

```
//
=====
//_____INITIAL VARIABLES (start
)
//
=====

var next = output.append(input[0]);
var xmlSRTE_Code_value_general = input[1];
var errList = '';
var dbInsertsXML = new XML(<message></message>);
var nodeCode_value_general = new XML(<Code_value_general></Code_value_general>);
var nodeV_Race_code = new XML(<V_Race_code></V_Race_code>);
var nodeLanguage_code = new XML(<Language_code></Language_code>);
var nodeV_State_county_code_value = new XML(<V_State_county_code_value></V_State_county_code_value>);
var nodeState_code = new XML(<State_code></State_code>);
var nodeLabtest_loinc = new XML(<Labtest_loinc></Labtest_loinc>);
var nodeLOINC_condition = new XML(<LOINC_condition></LOINC_condition>);
var nodeLOINC_code = new XML(<LOINC_code></LOINC_code>);
var nodeSnomed_condition = new XML(<Snomed_condition></Snomed_condition>);
var nodeSnomed_code = new XML(<Snomed_code></Snomed_code>);
var tempEle;

//
=====
//_____INITIAL VARIABLES (end
)
//
=====

//
=====
//_____FUNCTIONS (start
)
//
=====

//FUNCTION: fnCreateXMLEntry
//Function to create an XML node for checking codes and text.
function fnCreateXMLEntry (pathHL7, xmlNodeName, codeSetName) {
  var dbColumnName;
  var fieldData = next.getField(pathHL7);
  if (fieldData != null && fieldData != '') {
    tempEle = new XML(<query_record></query_record>);
```